TCP-Server 例程

TCP-Server 例程
- 例程简介
- 代码解析

TCP 协议
- TCP基础认知
  - 1.1 什么是TCP?
  - 1.2 TCP vs UDP
- TCP连接管理（图解三阶段）
  - 2.1 三次握手建立连接
  - 2.2 四次挥手终止连接
- TCP 可靠传输核心机制
  - 3.1 数据编号与确认
  - 3.2 超时重传
  - 3.3 滑动窗口协议
- 高级控制机制
  - 4.1 流量控制
  - 4.2 拥塞控制
  - 4.3 TCP请求报文头部结构
- 实际应用场景
- 常见问题解析

例程简介

本节我们介绍如何将K230作为TCP服务器使用

我们打开这一节的例程代码，修改代码中连接WIFI信息部分修改为自己家的WIFI或者手机热点

```
def connect_wifi():
    """
    连接WiFi并返回IP地址
    Connect to WiFi and return IP address
    """
    sta = network.WLAN(0)  # 创建WiFi站点对象 / Create WiFi station object
    sta.connect("WIFI名称", "WIFI密码")  # 连接到指定WiFi / Connect to specified
WiFi
    # 等待直到获取到IP地址 / Wait until IP address is obtained
    while sta.ifconfig()[0] == '0.0.0.0':
        os.exitpoint()
    return sta.ifconfig()[0]  # 返回IP地址 / Return IP address
```

然后点击左下角运行按钮，运行例程

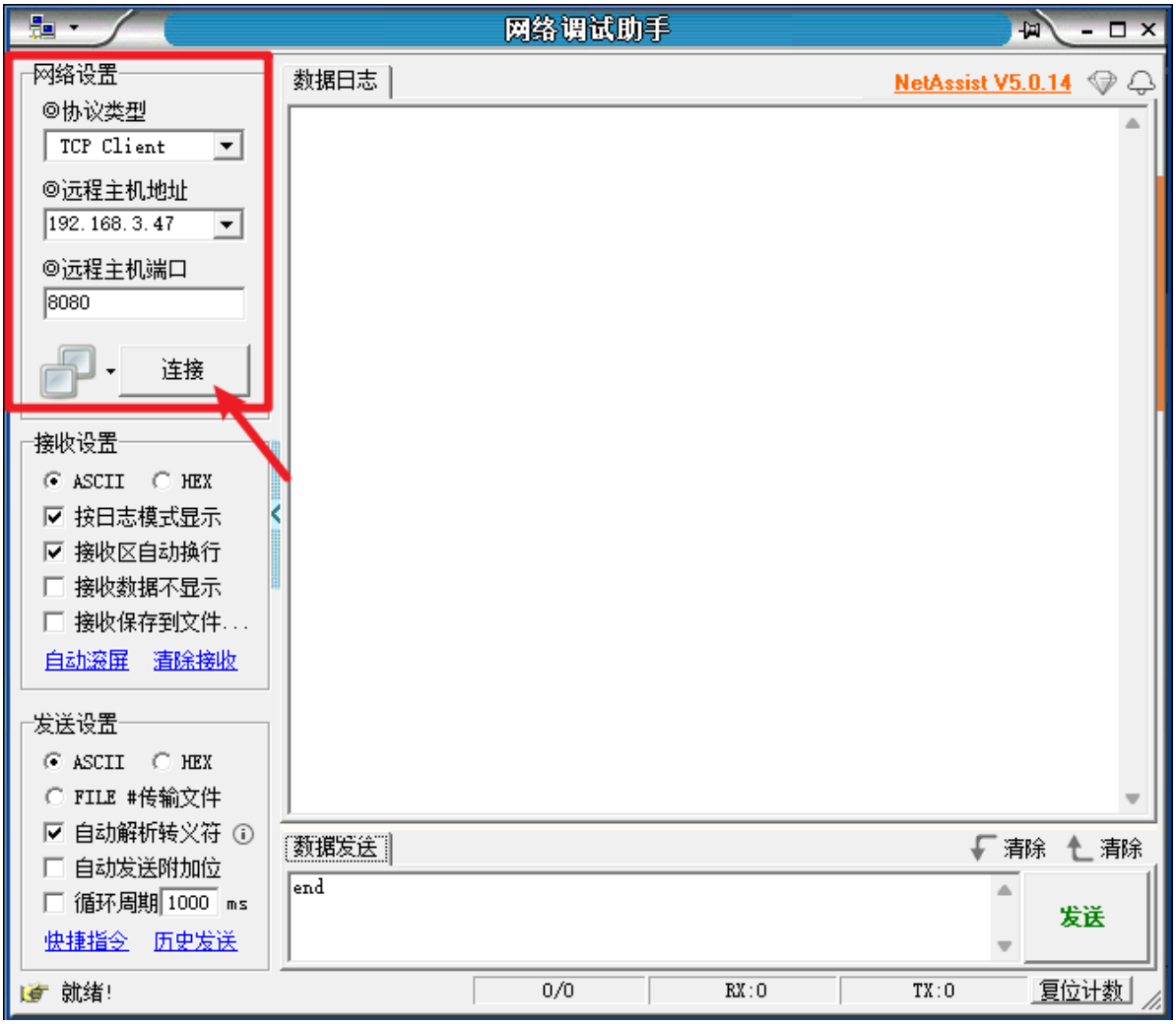
我们点开串行终端，等待输出服务器启动成功的消息

服务器启动成功后，串行终端会输出 TCP server running on xxxxxx:8080 (如果8080被占用了，可以换其他端口)

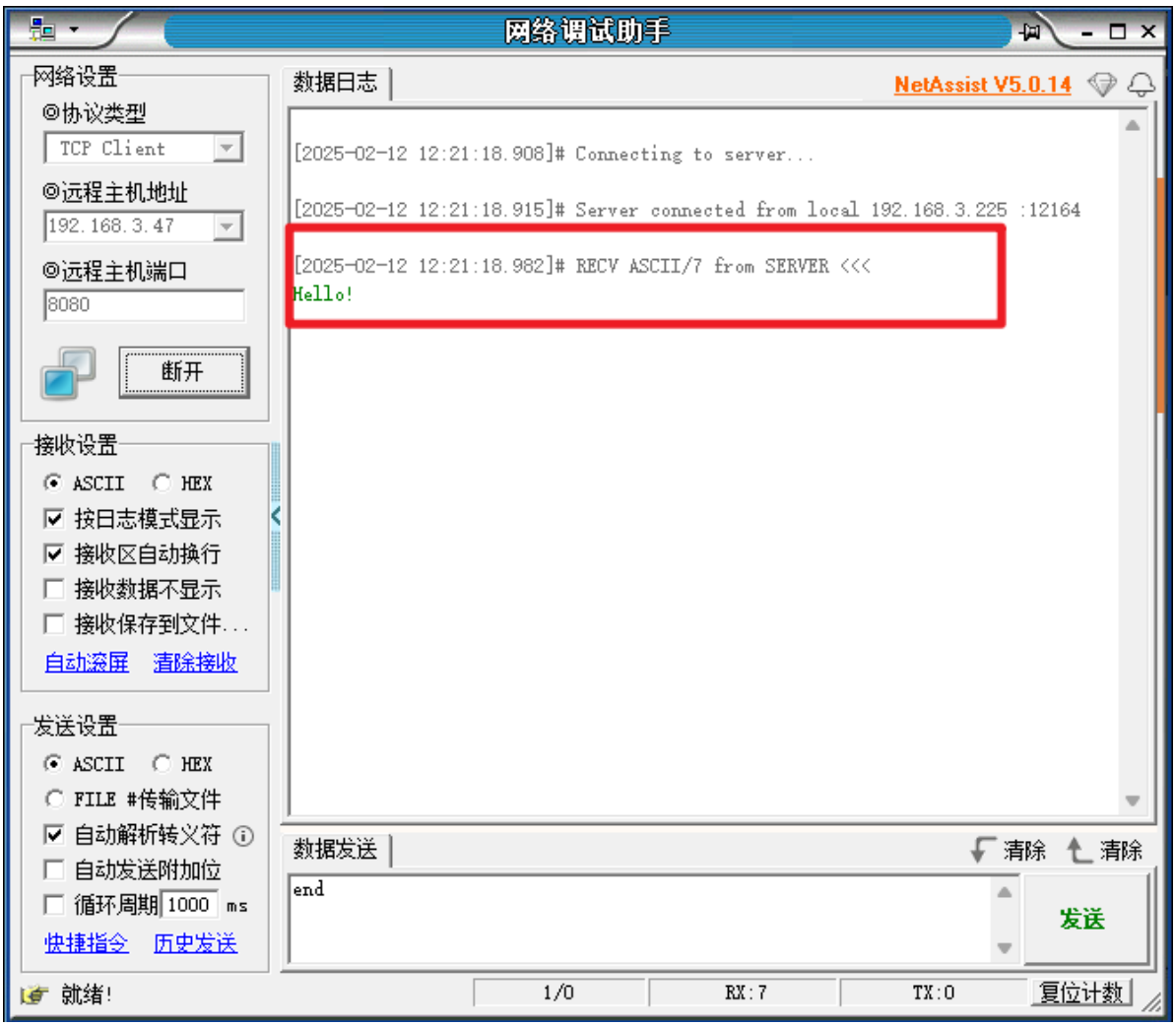
我这里是【TCP server running on 192.168.3.47:8080】

然后我们打开配套工具中的网络调试助手 NetAssist.exe

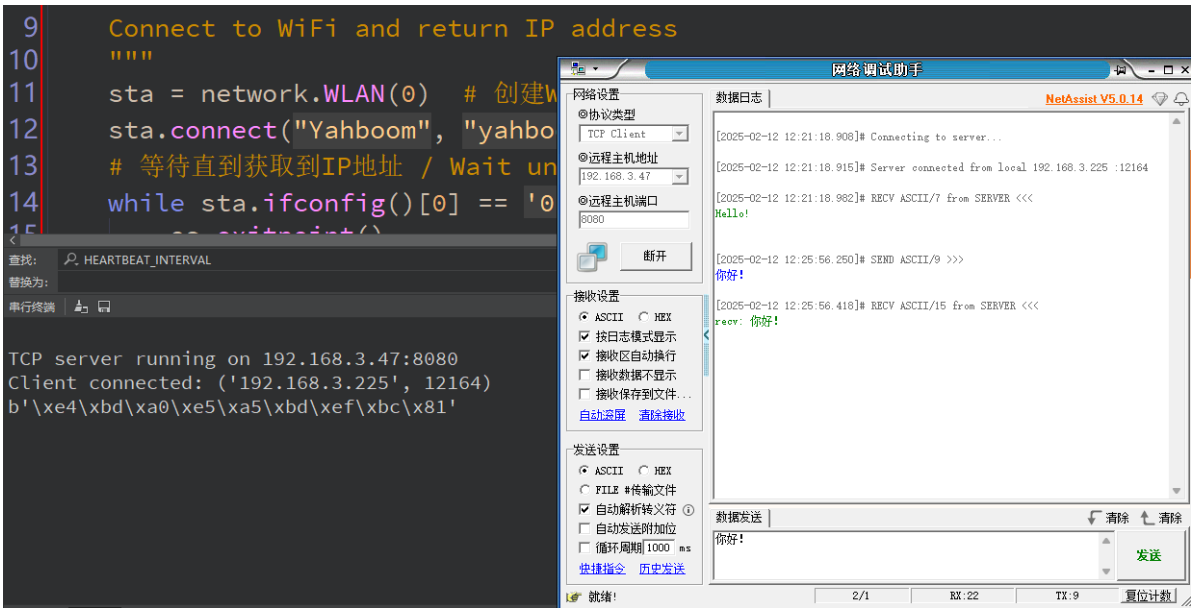
在左上角这个区域输入信息，然后点击连接



可以看到我们接收到了K230上运行的服务器发来的消息



我们在输入框里发送消息，服务端也能接收到我们发送过去的信息



> 由于编码问题，IDE输出的看起来像是乱码。实际上我们只需要调用解码功能，将print(data)改为print(data.decode())，就可以显示正确的内容了，此处留给大家自行尝试

代码解析

完整的代码和双语注释请参考 【源码汇总 / 11.Network / 03.tcp / tcp_server.py】

1. 导入模块部分：

```
import socket      # 导入socket模块，用于网络通信 / Import socket module for network communication
import network     # 导入network模块，用于WiFi连接 / Import network module for WiFi connection
import os          # 导入os模块，用于系统操作 / Import os module for system operations
import time        # 导入time模块，用于时间相关操作 / Import time module for time-related operations
```

这部分导入了程序所需的基础模块，包括网络通信、WiFi连接、系统操作和时间操作相关的模块。

2. WiFi连接函数：

```
def connect_wifi():
    """
    连接WiFi并返回IP地址
    Connect to WiFi and return IP address
    """
    sta = network.WLAN(0)  # 创建WiFi站点对象 / Create WiFi station object
    sta.connect("WIFI名称", "WIFI密码")  # 连接到指定WiFi / Connect to specified WiFi
    # 等待直到获取到IP地址 / Wait until IP address is obtained
    while sta.ifconfig()[0] == '0.0.0.0':
        os.exitpoint()
    return sta.ifconfig()[0]  # 返回IP地址 / Return IP address
```

这个函数负责：

- 创建WiFi站点对象
- 连接到指定的WiFi网络
- 等待直到获取到有效的IP地址
- 返回获取到的IP地址

3. 客户端关闭函数：

```
def close_client(client):
    """
    关闭客户端连接
    Close client connection
    """
    client.close()  # 关闭连接 / Close connection
    print("等待连接 ... Waiting for connect")
```

这是一个简单的辅助函数，用于关闭客户端连接并打印提示信息。

4. 主服务器函数：

```
def run_server():
    """
    运行TCP服务器的主函数
    Main function to run TCP server
    """
    # 连接WiFi并获取IP地址 / Connect to WiFi and get IP address
    ip = connect_wifi()

    # 创建TCP服务器 / Create TCP server
```

```
    server = socket.socket()
    # 设置端口复用 / Set port reuse
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    # 绑定IP和端口 / Bind IP and port
    server.bind((ip, 8080))
    # 开始监听，最大连接数为5 / Start listening, maximum 5 connections
    server.listen(5)
    print(f"TCP server running on {ip}:8080")
```

这部分代码完成了服务器的初始化：

- 连接WiFi并获取IP地址
- 创建TCP socket
- 设置端口复用（允许服务器重启时立即使用同一端口）
- 绑定IP和端口（8080）
- 开始监听连接，最大等待连接数为5

5. 主循环部分：

```
    while True:
        # 等待客户端连接 / Wait for client connection
        client, addr = server.accept()
        print(f"Client connected: {addr}")
        # 设置非阻塞模式 / Set non-blocking mode
        client.setblocking(False)

        # 发送欢迎消息 / Send welcome message
        client.write(f"Hello!\n".encode())

        while True:
            # 处理接收数据 / Handle received data
            try:
                data = client.read()
                if data:
                    print(data)
                    # 发送确认消息 / Send acknowledgment
                    client.write(b"recv: " + data)
                # 如果收到结束命令，关闭连接 / If end command received, close
connection
                if b"end" in data:
                    close_client(client)
                    break
            except:
                pass
```

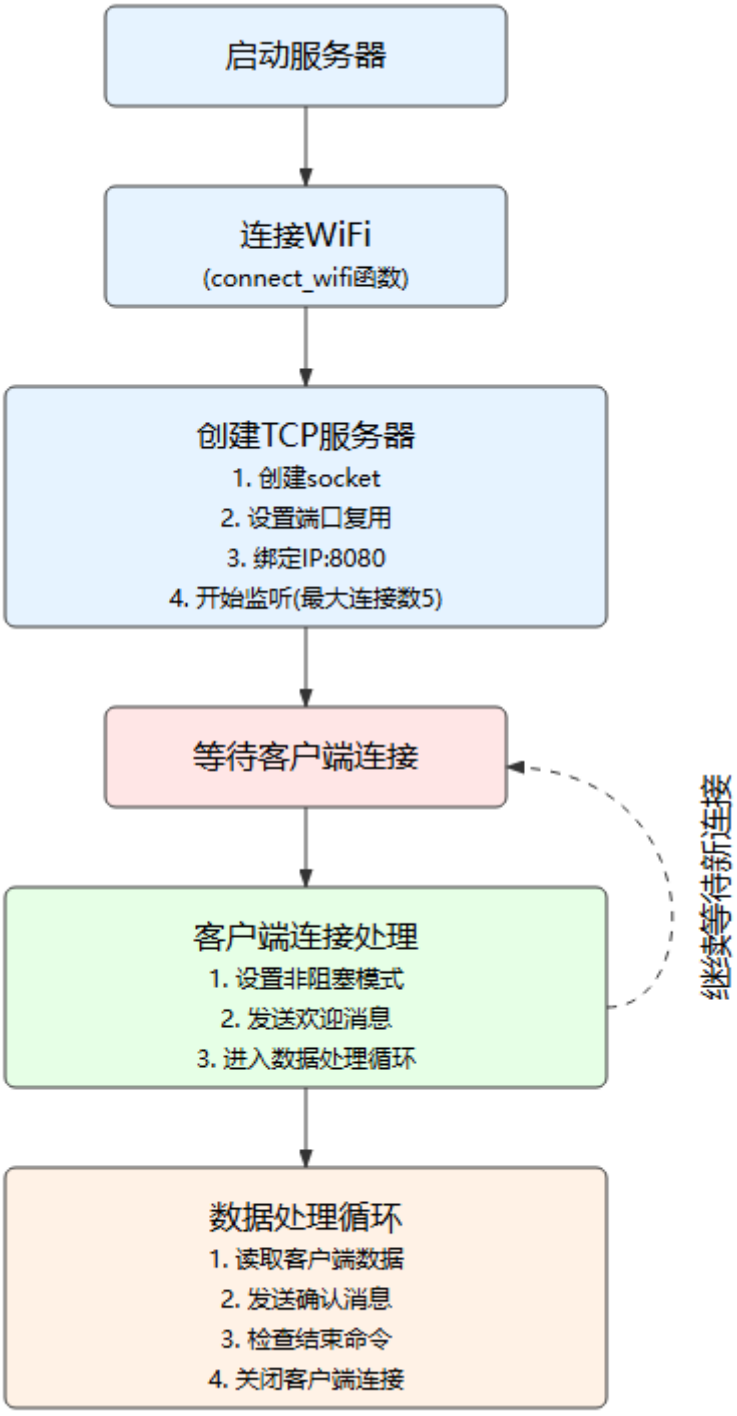
这是服务器的核心处理逻辑：

- 外层循环持续接受新的客户端连接
- 当有新连接时，设置为非阻塞模式
- 发送欢迎消息
- 内层循环处理与客户端的通信：
  - 读取客户端发送的数据
  - 如果收到数据，打印并发送确认消息
  - 如果收到"end"命令，关闭连接并跳出内层循环
  - 使用异常处理来处理非阻塞模式下的读取操作

6. 服务器启动：

```
# 启动服务器 / Start server
run_server()
```

程序流程图如图：



> 本例程中，客户端发送“end”可以正常断开与K230服务端的通信。但是如果直接断开连接，会造成服务端卡死。解决这一问题的方法可以参考本节的另一个附件源码 tcp_server_heartbeat.py

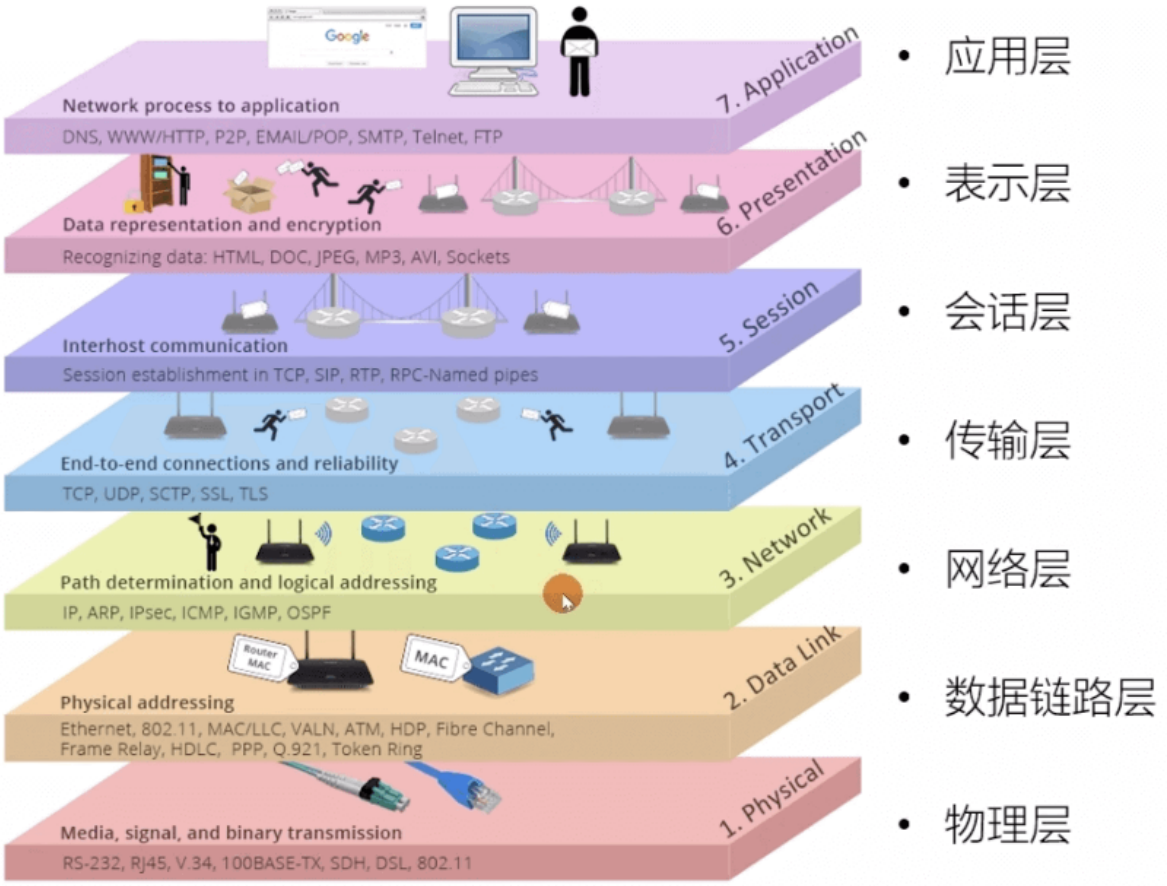
TCP 协议

我们在这里简单的描述一下TCP协议

TCP基础认知

1.1 什么是TCP?

- **定位**：传输层协议（OSI第4层）
- **特点**：面向连接、可靠传输、流量控制、拥塞控制
- **类比**：像快递公司的"保价包裹服务"（确保完整送达）



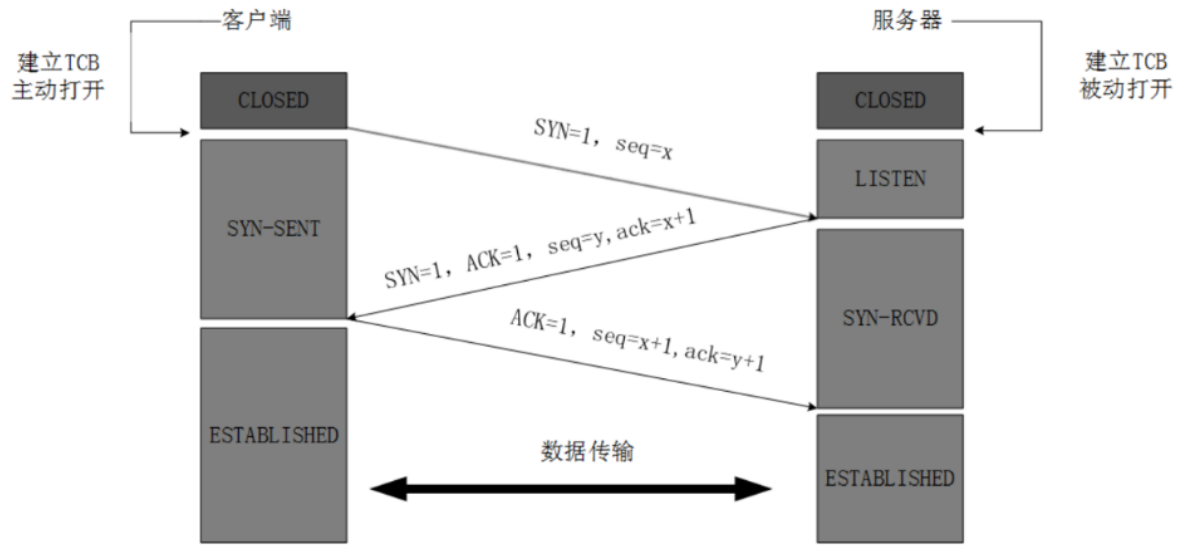
（图示描述：OSI模型中TCP位于IP层（网络层）之上，应用层之下）

1.2 TCP vs UDP

特性	TCP	UDP
连接方式	面向连接	无连接
可靠性	可靠传输	尽力而为
速度	较慢	极快
头部大小	20-60字节	8字节
典型应用	Web/Email	视频流/游戏

TCP连接管理（图解三阶段）

2.1 三次握手建立连接

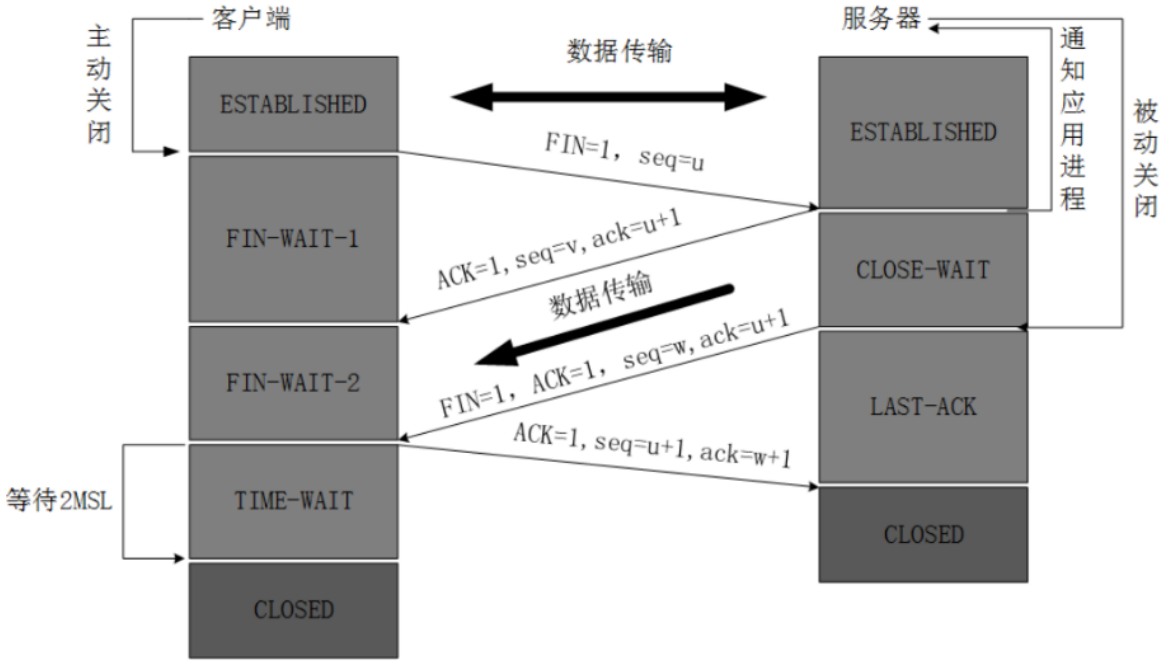


(图示描述：客户端与服务器通过SYN、SYN-ACK、ACK三次交互)

1. **SYN**（客户端→服务端）
   "我要建立连接，我的初始序列号是X"
2. **SYN-ACK**（服务端→客户端）
   "收到！我的初始序列号是Y，期待你下次发X+1"
3. **ACK**（客户端→服务端）
   "确认收到，接下来我们开始传数据吧！"

> 为什么不是两次握手?
> 防止已失效的连接请求突然到达导致资源浪费（网络延迟问题）

2.2 四次挥手终止连接



(图示描述：FIN和ACK的四个交互步骤)

1. **FIN**（A→B）
   "我要关闭连接了"

2. **ACK**（B→A）
   "收到关闭请求"
3. **FIN**（B→A）
   "我也准备好关闭了"
4. **ACK**（A→B）
   "确认关闭，连接终止"

> **TIME_WAIT状态**：主动关闭方等待2MSL时间，确保最后一个ACK被接收

TCP 可靠传输核心机制

3.1 数据编号与确认

- **序列号（Sequence Number）**：每个字节都有唯一编号
- **确认号（Ack Number）**："我已收到X之前的所有数据"
- **示例**：

```
发送方：发送[序列号=1，长度=100]的数据包
接收方：回复[ACK=101]（期待下一个数据包从101开始）
```

3.2 超时重传

- **RTO（Retransmission Timeout）**：动态计算的超时时间
- **快速重传**：收到3个重复ACK立即重传

3.3 滑动窗口协议

- **窗口大小**：接收方通告的可用缓冲区大小
- **动态调整**：根据网络状况实时变化

高级控制机制

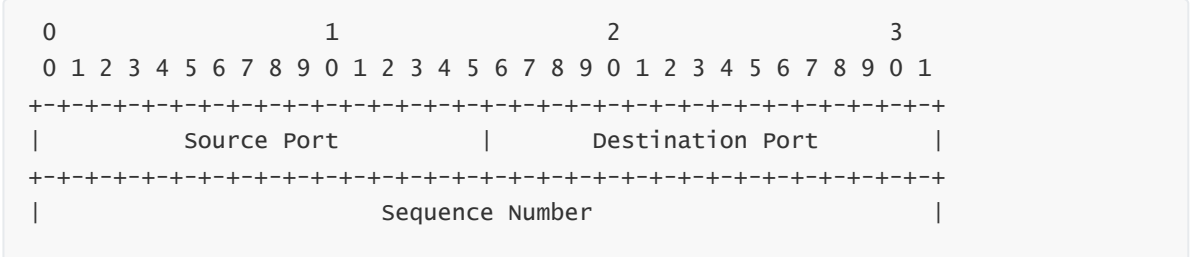
4.1 流量控制

- **原理**：通过接收窗口（rwnd）控制发送速率
- **零窗口探测**：当接收窗口为0时发送探测包

4.2 拥塞控制

1. **慢启动**：指数增长直到阈值
2. **拥塞避免**：线性增长
3. **快速恢复**：遇到丢包时减半窗口

4.3 TCP请求报文头部结构



```
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                     Acknowledgment Number                     |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|  Data |           |U|A|P|R|S|F|                               |
| Offset| Reserved  |R|C|S|S|Y|I|            Window             |
|       |           |G|K|H|T|N|N|                               |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           Checksum            |         Urgent Pointer        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                    Options (if Data Offset > 5)               |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                             Data                              |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

实际应用场景

1. **HTTP通信**：每个网页加载都经历TCP连接
2. **文件传输**：FTP等协议依赖TCP可靠性
3. **数据库连接**：MySQL等数据库默认使用TCP

常见问题解析

Q1：TCP粘包问题如何处理?
通过应用层协议设计（如添加长度头或分隔符）

Q2：为什么有些视频直播用TCP?
当画质优先时使用TCP，实时性优先用UDP+QUIC

Q3：如何保持长连接?
使用Keep-Alive机制，定时发送探测包

> 本文中提到的附件源码 tcp_server_heartbeat.py 就是参考了Keep-Alive机制